UNIT-5
FUNCTIONS and MODULES

Functions:

- In Python, function is a group of related statements that perform a specific task.
- Functions help break our program into smaller and modular chunks. As our program grows larger and larger, functions make it more organized and manageable.
- Furthermore, it avoids repetition and makes code reusable.
- We have two types of functions
    1. Built-in functions
    2. User-defined functions

Built-in functions:

- The Python interpreter has a number of functions that are always available for use. These functions are called built-in functions.
- For example, print() function prints the given object to the standard output device (screen) or to the text stream file.
- In Python 3.6 (latest version), there are 68 built-in functions.
- **Examples are:** print(),type(),id(),min(),max(),input(),sqrt(),factorial(),……etc.

```
import math
print(max([1,2,3,4,5]))#5
print(math.sqrt(144))#12
print(math.factorial(5))#120
```

User-defined functions:

- Functions that we define ourselves to do certain specific task are referred as user-defined functions.

Creating or Defining a function:

- We can create our own functions by using **def** keyword.
- Next statement after def statement is a docstring about function. Docstring describes about function what the function does.
- Add docstrings as a triple quoted string beginning with the first line of a function or method.
- We can return a value from function using **return** statement this is optional.

  Syntax:

```
def function_name(parameters):
      """docstring"""
      statement(s)
```

- **Example:**

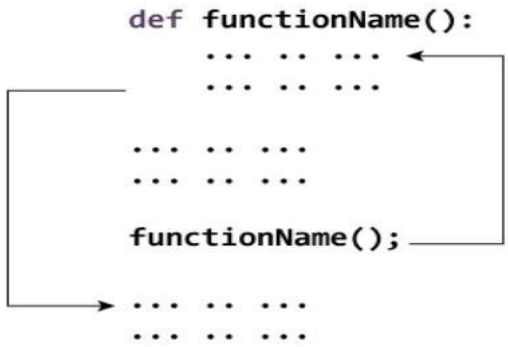
```
def greet(name):
      """This function greets to
      the person passed in as parameter"""
      print("Hello, " + name + ". Good morning!")
```

Calling a function:

- Once we have defined a function, we can call it from another function, program or even the Python prompt. To call a function we simply type the function name with appropriate parameters.

```
>>> greet('Paul')
Hello, Paul. Good morning!
```

- Function working flow



- We can print the documentation of the function using **_doc_** keyword.

```
print(greet.__doc__)
Output:
This function greets to
     the person passed into the name parameter
```

- **Example-1:**
  Program: add.py

```
def add_numbers(x,y):
     """This function adds two numbers and return the result"""
     sum = x + y
     return sum
num1 = 5
num2 = 6
print("The sum is", add_numbers(num1, num2))
```

  Output:

```
The sum is 11
```

- **Example-2:** Python Program to find the factors of a number
  Program: factors.py

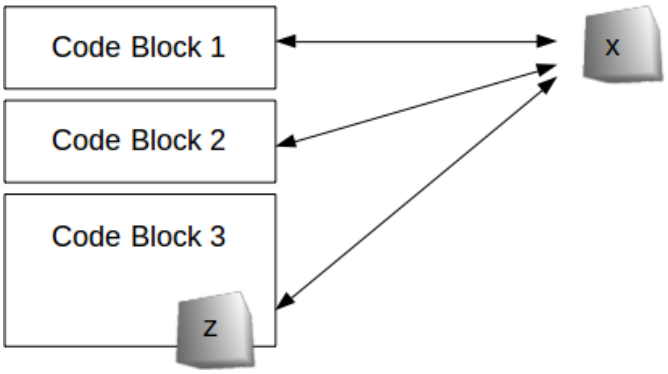
```
def print_factors(x):
     print("The factors of",x,"are:")
     for i in range(1, x + 1):
          if x % i == 0:
               print(i,end=" ")
num = int(input("Enter a number: "))
print_factors(num)
```

  Output:

```
Enter a  number:  16
The factors of 32 are:
1 2 4 8 16
```

Global and Local Variables:

- There are two types of variables: global variables and local variables.
- A global variable can be reached anywhere in the code, a local only in the scope.
- Variables can only reach the area in which they are defined, which is called scope. Think of it as the area of code where variables can be used. Python supports global variables (usable in the entire program) and local variables.
- By default, all variables declared in a function are local variables. To access a global variable inside a function, it's required to explicitly define 'global variable'.



- A global variable (x) can be reached and modified anywhere in the code, local variable (z) exists only in block 3.

Local variables:

- Local variables can only be reached in their scope.The example below has two local variables: x and y.

```
def sum(x,y):
      sum = x + y
      return sum
print(sum(8,6))#14
```

- The variables x and y can only be used inside the function sum, they don't exist outside of the function.
- Local variables cannot be used outside of their scope, this line will not work:

```
def numbers(x,y):
      x=10
      y=20
      print(x,y)#10 20
print(x,y) #gives an error
```

Global variables

- A global variable can be used anywhere in the code. In the example below we define a global variable z.

```
z = 10
def afunction():
      global z
      print(z)#10
afunction()
print(z)#10
```

- The global variable z can be used all throughout the program, inside functions or outside.
- A global variable can modified inside a function and change for the entire program:

```
z = 10
def afunction():
      global z
      print(z)#10
      z = 9
afunction()
print(z)#9
```

- After calling afunction(), the global variable is changed for the entire program.

Recursion in python:

- In Python, a function is recursive if it calls itself and has a termination condition.
- **Factorial with recursion:**
  The mathematical definition of factorial is: n! = n * (n-1)!, if n > 1 and f(1) = 1. Example: 3! = 3 x 2 x 1 = 6. We can implement this in Python using a recursive function:

```
def factorial(n):
      if n == 1:
            return 1
      else:
            return n * factorial(n-1)
print(factorial(3))
```

  When calling the factorial function n = 3. Thus it returns n * factorial(n-1). This process will continue until n = 1. If n==1 is reached, it will return the result.
- **Limitation of recursion is:**
  Every time a function calls itself and stores some memory. Thus, a recursive function could hold much more memory than a traditional function. Python stops the function calls after a depth of 1000 calls.

Function Arguments:

- In python function will take arguments or parameters.
- **Example:**

```
def greet(name,msg):
      print("Hello",name + ', ' + msg)
greet("Raju","Good morning!")
```

Output:

```
Hello Raju Good morning!
```

- Here, the function greet() has two parameters.
- Since, we have called this function with two arguments, it runs smoothly and we do not get any error.
- If we call it with different number of arguments, the interpreter will complain. Below is a call to this function with one and no arguments along with their respective error messages.

```
>>> greet("Bharath")    # only one argument
TypeError: greet() missing 1 required positional argument: 'msg'

>>> greet()    # no arguments
TypeError: greet() missing 2 required positional arguments: 'name'
and 'msg'
```

- **Variable Function Arguments:**
  - In Python there are other ways to define a function which can take variable number of arguments.
  - Three different forms of this type are described below.
    1. Default arguments
    2. Keyword arguments
    3. Arbitrary arguments
  - **Default arguments:**

    Function arguments can have default values in Python.We can provide a default value to an argument by using the assignment operator (=). Here is an example.

    If we provide default value, then at the time of function call that argument is optional i.e. called as **optional parameter**. Also we can change default argument value at the time of function call.

```
def greet(name, msg = "Good morning!"):
      print("Hello",name + ', ' + msg)
greet("Raju")
greet("Rani","How are you?")
```

Output:

```
Raju Good morning!
Rani How are you?
```

- **Keyword arguments:**

  Python allows functions to be called using keyword arguments. When we call functions in this way, the order (position) of the arguments can be changed. Following calls to the above function are all valid and produce the result.

  # 2 keyword arguments
  ```
  >>>greet(name = "Bruce",msg = "How do you do?")
  ```
  # 2 keyword arguments (out of order)
  ```
  >>>greet(msg = "How do you do?",name = "Bruce")
  ```
  # 1 positional, 1 keyword argument
  ```
  >>>greet("Bruce",msg = "How do you do?")
  ```

  As we can see, we can mix positional arguments with keyword arguments during a function call. But we must keep in mind that keyword arguments must follow positional arguments.Having a positional argument after keyword arguments will result into errors. For example the function call as follows:

  ```
  >>>greet(name="Bruce","How do you do?")
  SyntaxError: non-keyword arg after keyword arg
  ```

- **Arbitrary arguments:**

  Arbitrary arguments means we can pass multiple values for single argument by placing asterisk (*) before the argument.

  ```
  def greet(*names):
        for name in names:
                    print("Hello",name)
  greet("Ramu","Seetha","Krishna","Radha")
  ```
  Output:
  ```
  Hello Ramu
  Hello Seetha
  Hello Krishna
  Hello Radha
  ```

Passing collections to function:

In python collections are nothing but list, tuple, set and dictionaries. So we can pass collections as arguments to functions.For example passing list as argument.

```
def change_list(nums):
    val=10;
    new_list=[]
    for i in range(1,len(nums)):
        new_list.append(nums[i]+val)
    print('After function call list',new_list)
my_list=[1,2,3,4,5]
print('Before Function call list:',my_list)
change_list(my_list)
```
output:
```
Before Function call list: [1, 2, 3, 4, 5]
After function call list [12, 13, 14, 15]
```

Passing Function to a Function:

- We can pass function as argument to another function. for example

```
def value(r):
          return r
def circle_area(f):
          area=3.14*(f**2)
          return area
res=circle_area(value(4))
print('Area of circle is:',res)
```

output:

```
Area of circle is: 50.24
```

Nested function or Inner Function:

- Like nested loops, we can also have nested functions in Python. We simply create a function using def inside another function to nest two functions.
- **For example,**

```
def f1(): #outer function
     print ("Hello")
     def f2(): #inner function
          print ("world")
     f2()
f1()
```

Output:

```
Hello
World
```

- **Nonlocal Variable:**
  We can use the nonlocal keyword to change the value of the variable of the outer function similar to using global keyword to change the value of global variables.

```
def f1(): #outer function
    a = 1
    def f2(): #outer function
        nonlocal a
        a = 2
        print (a) #prints 2
    f2()
    print (a) #prints 2
f1()
```

Anonymous function:

- In Python, anonymous function is a function that is defined without a name.
- While normal functions are defined using the def keyword, in Python anonymous functions are defined using the lambda keyword.
- Hence, anonymous functions are also called lambda functions.
- A lambda function in python has the following syntax.

```
lambda arguments: expression
```

- Lambda functions can have any number of arguments but only one expression. The expression is evaluated and returned. Lambda functions can be used wherever function objects are required.
- **Example-1:**
  ```
  double = lambda x: x * 2
  print(double(5))#10
  ```
- **Example-2:**
  ```
  square = lambda x: x ** 2
  print(square(5))#25
  ```

- In Example-1, lambda x: x * 2 is the lambda function. Here x is the argument and x * 2 is the expression that gets evaluated and returned.
- This function has no name. It returns a function object which is assigned to the identifier double. We can now call it as a normal function.
- The statement `'double = lambda x: x * 2'` is nearly the same as
  ```
  def double(x):
      return x * 2
  ```

<u>Use of Lambda Function in python:</u>

- We use lambda functions when we require a nameless function for a short period of time.
- In Python, we generally use it as an argument to a higher-order function (a function that takes in other functions as arguments).
- Lambda functions are used along with built-in functions like filter(), map(),..etc.
- **<u>filter():</u>**
  - The filter() function in Python takes in a function and a list as arguments.
  - The function is called with all the items in the list and a new list is returned which contains items for which the function evaluats to True.
  - Here is an example use of filter() function to filter out even numbers from a list.

    ```
    my_list = [1, 5, 4, 6, 8, 11, 3, 12]
    new_list = list(filter(lambda x: (x%2 == 0) , my_list))
    print(new_list)
    ```
    Output: `[4, 6, 8, 12]`

- **<u>map():</u>**
  - The map() function in Python takes in a function and a list.
  - The function is called with all the items in the list and a new list is returned which contains items returned by that function for each item.
  - Here is an example use of map() function to double all the items in a list.

    ```
    my_list = [1, 5, 4, 6, 8, 11, 3, 12]
    new_list = list(map(lambda x: x * 2 , my_list))
    print(new_list)
    ```
    Output: [2, 10, 8, 12, 16, 22, 6, 24]

Modules:

- A module is a file containing Python definitions and statements. A module can define functions, classes and variables. A module can also include runnable code. Grouping related code into a module makes the code easier to understand and use.
- **Example:A simple user defined module, calc.py**

```
def add(x, y):
    return (x+y)
def subtract(x, y):
    return (x-y)
```

- **Importing a module:**

  We can import modules into another program with the help of **import** keyword

  Example: calculator.py

```
import calc
print(calc.add(10,20)) #30
print(calc.subtract(30,20)) #10
```

Possible ways to import a module:

Ways	Form	Example
Import statement	import module_name	import math print(math.pi) # 3.141592653589793
Import with renaming	import module_name as mn	import math as m print(m.pi) # 3.141592653589793
from…import format	from module_name import attributes	from math import pi,e print(pi) # 3.141592653589793 print(e) # 2.718281828459045
Import all names	from module_name import *	from math import * print(pi) # 3.141592653589793 print(sqrt(4)) #2 print(factorial(5)) #120

Standard Modules:

Python has different modules for different type of applications. Following are some standard modules.

- sys
- math
- time
- random

sys-module:

- This module provides access to some variables used or maintained by the interpreter and to functions that interact strongly with the interpreter
- Importing sys module as bellow

```
import sys
```

- The sys module allows you to use stdin() and stdout(), as well as stderr().

```
import sys
print('enter any value:')
val=sys.stdin.read()
sys.stdout.write(val)
sys.stderr.write('to display error')
```

- Also we can utilize sys.argv().The idea of sys.argv is to allow you to pass arguments through to Python from the command line.
- With stdout and stdin, we can pass messages and errors through to the command line, or just use it for logging purposes.

```
import sys
print('The command line arguments are:')
for i in sys.argv:
     print(i)
```

- For passing command line arguments run the program in command prompt

```
F:\Python Content\Python programs>py cmdargs1.py hello 123
The command line arguments are:
cmdargs1.py
hello
123
```

- sys module has some variables also, like

sys.byteorder: An indicator of the native byte order.
sys.copyright: A string containing the copyright pertaining to the Python interpreter.
sys.path: A list of strings that specifies the search path for modules.
sys.modules: This is a dictionary that maps module names to modules which have already been loaded.
sys.platform: This string contains a platform identifier
sys.version: A string containing the version number of the Python interpreter
sys.winver: The version number used to form registry keys on Windows platforms.

math-module:

- The math module is a standard module in Python and is always available. To use mathematical functions under this module, you have to import the module using import math.

```
import math
math.sqrt(4)#2
```

- Some of the math functions and variables listed bellow

Function	Description
ceil(x)	Returns the smallest integer greater than or equal to x.
factorial(x)	Returns the factorial of x
floor(x)	Returns the largest integer less than or equal to x
exp(x)	Returns e**x
log(x[, base])	Returns the logarithm of x to the base (defaults to e)
log2(x)	Returns the base-2 logarithm of x
log10(x)	Returns the base-10 logarithm of x
pow(x, y)	Returns x raised to the power y
sqrt(x)	Returns the square root of x
cos(x)	Returns the cosine of x
sin(x)	Returns the sine of x
tan(x)	Returns the tangent of x
degrees(x)	Converts angle x from radians to degrees
radians(x)	Converts angle x from degrees to radians
Pi	Mathematical constant, the ratio of circumference of a circle to it's diameter (3.14159...)
E	mathematical constant e (2.71828...)

Example program:

```
from math import *
# constants in math module
print('mathematical constant π(pi) is:',pi)=
print('mathematical constant e is:',e)
print('mathematical constant τ(tau) is:',tau)

#Number-theoretic and representation functions
print('Ceil value of 76.56:',ceil(76.56))
print('Floor value of 76.56:',floor(76.56))
print('factorial of 5:',factorial(5))
print('GCD of 4,8:',gcd(4,8))

#Power and logarithmic functions
print('Exponential value of 2:',exp(2))
print('value of log(2) is:',log(2))
print('value of log(2) base 10 is:',log(2,10))
print('value of log10(10) is:',log10(10))
print('pow(10.25,2) is:',pow(10.25,2))
print('Square root of 144 is:',sqrt(144))
```

Output:

```
mathematical constant π(pi) is: 3.141592653589793
mathematical constant e is: 2.718281828459045
mathematical constant τ(tau) is: 6.283185307179586
Ceil value of 76.56: 77
Floor value of 76.56: 76
factorial of 5: 120
GCD of 4,8: 4
Exponential value of 2: 7.38905609893065
value of log(2) is: 0.6931471805599453
value of log(2) base 10 is: 0.30102999566398114
value of log10(10) is: 1.0
pow(10.25,2) is: 105.0625
Square root of 144 is: 12.0
```

****

time-module:

- Time is the primary python module to program date and time. In Python, time is an object, not a plain string or a timestamp. So to handle them in a program, we need to import the respective module or the class from the module.
- Python time module provides the ability to read, represent, and reset the time information in many ways.
  - time functions: time.time(), time.clock(), time.ctime(), time.sleep(), time.strftime()
  - Timezone: time.timezone property, time.tzname property

- This module follows the EPOCH convention which refers to the point where the time starts. It supports the dates and times from the epoch to the year 2038.

- The Unix EPOCH is midnight on 12:00 am January 1, 1970. To determine the EPOCH value on your system, use the following code.

```
>>> import time
>>> time.gmtime(0)
time.struct_time(tm_year=1970,   tm_mon=1,   tm_mday=1,   tm_hour=0,
tm_min=0, tm_sec=0, tm_wday=3, tm_yday=1, tm_isdst=0)
```

Note – The function time.gmtime() returns a struct_time object.

- Some time functions return time in DST format. DST is the Daylight-Saving-Time. It is a mechanism which moves the clocks forward by 1 hour during the summer time, and back again in the fall.

Useful Functions In Python Time Module.

Following are the most used functions and the data structure of the time module in Python which you must know.

- time.time()
- time.clock()
- time.ctime()
- time.sleep()
- time.strftime()

time.time():

- The time() is a core function of the time module. It measures the number of seconds since the epoch as a floating point value.
- **Syntax:** `time.time()`
- **Example1:**

```
import time
print('The current time is:', time.time())
```

Output - `The current time is: 1504634843.1003768`

- Another common application of Python time function is to measure the elapsed Wall-clock time between two points.
- **Example2** – time() to measure the time elapsed (Wall Clock Time)

```
import time
start = time.time()
print("I'm doing a job.")
time.sleep(0.1)
end = time.time()
print(end - start)
```

time.clock():

- Unlike the time.time() function which computes the wall clock time, the time.clock() returns the processor clock time.

  Syntax: `time.clock()`
- We can use time.clock() for performance testing/benchmarking and time.time() to schedule something.The output of the clock() function reflects the right time taken by the program.
- time.clock() has been deprecated in Python 3.3

time.ctime():

- It takes the time in "seconds since the epoch" as input and translates into a human readable string value as per the local time. In case of no argument, it calls the time() function and returns the current time.

  Syntax: `time.ctime(seconds)`
- **Example:**

```
import time
print('The current time is :', time.ctime())
newtime = time.time() + 20
print('20 secs from now :', time.ctime(newtime))
```

Output:

```
The current time is : Sun Sep 10 00:02:41 2017
20 seconds from now : Sun Sep 10 00:03:01 2017
```

- Note that the length of the string returned by time.ctime() is 24 characters.

time.sleep():

- It halts the execution of the current thread for the specified number of seconds. You can pass a floating point value as the argument to register a more precise sleep time.

  Syntax: `time.sleep(seconds)`
- With the sleep() function, our program/script can pause for a given number of seconds.
- It is a useful function in cases like when I need to wait for a file to finish closing or let a database commit to happen.
- **Example:**

```
import time
# using ctime() to display present time
print ("Begin Execution : ",end="")
print (time.ctime())
# using sleep() to suspend execution
print ('Waiting for 5 sec.')
time.sleep(5)

# using ctime() to show present time
print ("End Execution : ",end="")
print (time.ctime())
```

Output:

```
Begin Execution : Sun Sep 10 00:27:06 2017
Waiting for 5 sec.
End Execution : Sun Sep 10 00:27:12 2017
```

- **Note** – In Python 3.5, the sleep() function continues irrespective of the interrupt caused by a signal. However, it breaks if the signal handler threw an exception.

time.strftime():

- This function takes a tuple or struct_time in the second argument and converts to a string as per the format specified in the first argument. If t is not present, then it calls the localtime() to use the current date.

  Syntax: `time.strftime(format[, t])`
- **Example:**
  ```
  import time
  now = time.localtime(time.time())
  print(time.asctime(now))
  print(time.strftime("%y/%m/%d %H:%M", now))
  print(time.strftime("%a %b %d", now))
  print(time.strftime("%c", now))
  print(time.strftime("%I %p", now))
  print(time.strftime("%Y-%m-%d %H:%M:%S %Z", now))
  ```
  Output:
  ```
  Sun Sep 10 01:36:28 2017
  17/09/10 01:36
  Sun Sep 10
  Sun Sep 10 01:36:28 2017
  01 AM
  2017-09-10 01:36:28 India Standard Time
  ```
- In the first few lines, we are calling two time-related functions. The localtime() function creates a struct_time object using the time argument. And the asctime() function converts a struct_time object into a time string.

Time zones in python:

- Time zone is a term commonly used with time. It represents a region where same standard time is used for legal, commercial and social purposes. There are 24 time zones in the world.
- **UTC** is an acronym for Coordinated Universal Time. And it is a time standard, not a time zone by which the world regulates clocks and time.
- The **GMT** refers to Greenwich Mean Time and IST is a term used to represent Indian Standard Time. Both are time zones.
- **Note** – GMT and IST contrast by +5.30 hours. Also, IST applies to India whereas GMT/UTC is applicable for the world.
- There are two time-properties which give you the timezone info.
  time.timezone – It returns the offset of the local (non-DST) timezone in UTC format.
  ```
  >>> time.timezone
  -19800
  ```
  time.tzname – It returns a tuple containing the local non-DST and DST time zones.
  ```
  >>> time.tzname
  ('India Standard Time', 'India Daylight Time')
  ```

- **Setting Timezone:**
  time.tzset():It resets the Timezone for all library functions and uses the environment variable TZ to set the new value.The limitation is that it works only on the Unix based platforms.
  Example:
  ```
  # Set timezone on a unix system
  import os, time
  print('Current timezone is', time.tzname)
  os.environ['TZ'] = 'Europe/London'
  time.tzset()
  print('New timezone is', time.tzname)
  ```
  Output
  Python 3.6.1 (default, Dec 2015, 13:05:11)[GCC 4.8.2] on linux
  Current timezone is ('UTC', 'UTC')
  New timezone is ('GMT', 'BST')

****

random-module:

- With Python random module, we can generate random numbers to fulfill different programming needs.
- It has a no. of functions like randint(), random(), choice(), uniform() that as a programmer we can decide to use depending on the use case.

Generate Random Integers:

Following three functions allow generating random integers in Python.

randrange():

This function has following two variations.

randrange(stop):

- It will produce a random integer value less than the value specified by the [stop] argument. If "r" is a random number, then its value will lie in the range 0 <= r < stop.
  Syntax: `random.randrange(stop)`
- [stop] – It is the boundary value of the range to be used for generating the random number. Pass a valid integer value only. Default value is zero if not passed.
- **Example:**
  ```
  import random
  print(random.randrange(999))
  ```
  Output:
  ```
  856
  ```

randrange(Start, Stop[, Step]):

- It uses the following range [start, stop-1] to return a uniquely selected integer value. If the [step] is specified, then the randrange() output is incremented by it.
- If "r" is a random number, then its value will lie in the range start <= r < stop.
  Syntax: `random.randrange(start, stop[, step])`
- [start] – It is the base value of the range and may fall in the selection.
- [stop] – It is the boundary value of the range excluding from the selection
- [step] – It is the value with which the number is incremented. The default value is 0 if not passed.

- **Example:**

```
import random

# Generate random integer in the interval [4,10]
print(random.randrange(4,11))

# Generate random integer in [4,10] with increment of 3
print(random.randrange(4, 11, 3))

# Generate random even numbers between 2 and 24
print(random.randrange(2, 25, 2))

# Generate random even numbers between 1 and 25
print(random.randrange(1, 26, 2))
```

Output:

```
#1.  #2.  #3.
10   7    4
4    10   4
4    12   8
17   3    23
```

randint(low, high):

- The randint() function is one of many functions which handle random numbers. It has two parameters low and high and generates an integer between low and high (including both).
- **Example:**

Generate random integers in range 0 through 9.

```
import random
iter = 0
while iter < 10:
      # Get random number in range 0 through 9.
      r = random.randint(0, 9)
      print(r)
      iter += 1
```

Output:

4
3
0
1
5
7
9
3
0
8

choice(seq):

- The choice() function arbitrarily determines an element from the given sequence.
- **Note**- A sequence in Python is the generic term for an ordered set like a list, tuple etc.
- **Example:**

```
import random
# Generate a random string from the list of strings
print(random.choice( ['Python', 'C++', 'Java'] ))

# Generate a random number from the list [-1, 1, 3.5, 7, 15]
print(random.choice([-1, 1, 3.5, 9, 15]))

# Generate a random number from a uniformly distributed tuple
print(random.choice((1.1, -5, 6, 4, 7)))

# Generate a random char from a string
print(random.choice('Learn Python Programming'))
```

shuffle(list):

- The shuffle() function rearranges the items of a list in place so that they occur in a random order.
- For shuffling, it uses the Fisher-Yates algorithm which has O(n) complexity. It starts by iterating the last element in the array to the first entry, then swap each entry with an entry at a random index below it.
- **Example:**

```
from random import shuffle
mylist = [11,21,31,41,51]
shuffle(mylist)
print(mylist)
```

Output:

```
#1. [41, 51, 11, 31, 21]
#2. [11, 31, 41, 51, 21]
```

sample(collection, random list length):

- The sample() function randomly selects N items from a given collection (list, tuple, string, dictionary, set) and returns them as a list.
- It works by sampling the items without replacement. It means a single element from the sequence can appear in the resultant list at most once.
- **Example:**

```
from random import sample
# Select any three chars from a string
print(sample('Python',3))

# Randomly select a tuple of three elements from a base tuple
print(sample((21, 12, -31, 24, 65, 16.3), 3))

# Randomly select a list of three elements from a base list
print(sample([11, 12, 13, 14, -11, -12, -13, -14], 3))

# Randomly select a subset of size three from a given set of numbers
print(sample({110, 120, 130, 140}, 3))
```

```
# Randomly select a subset of size three from a given set of strings
print(sample({'Python', 'C++', 'Java', 'Go'}, 3))
```

Output:

```
['y', 'o', 't']
[21, 12, 24]
[-14, 14, -13]
[140, 110, 130]
['Python', 'Java', 'C++']
```

random():

- It selects the next random floating point number from the range [0.0, 1.0].
- It is a semi-open range as the random function will always return a decimal no. less than its upper bound. However, it may return 0.
- **Example:**

```
import random
# Generate a floating-point pseudo-random number between 0 and 1.
print(random())
```

Output:

```
#1. 0.4968601882231284
#2. 0.831505293496292
```

uniform(lower, upper):

- It is an extension of the random() function. In this, you can specify the lower and upper bounds to generate a random number other than the ones between 0 and 1.
- **Example-1:**

```
# uniform() method to generate floating-point random numbers?
import random
lower = 111; upper = 999
random_float = random.uniform(lower, upper)
print(random_float)
```

Output:

```
#1. 466.63369297787557
#2. 315.3719206118211
```

- **Example-2:**

```
# Generate a floating-point random number with fixed precision
import random
lower = 1.0; upper = 2.0; fixed_precision = 2
random_float = random.uniform(lower, upper)
print(round(random_float, fixed_precision))
```

Output:

```
#1. 1.48
#2. 1.69
#3. 1.57
```

****

dir() function:

- dir() is a powerful inbuilt function in Python3, which returns list of the attributes and methods of any object (say functions , modules, strings, lists, dictionaries etc.)

```
Syntax :dir({object})
Parameters : object [optional] : Takes object name
```

- **Returns :** dir() tries to return a valid list of attributes of the object it is called upon. Also, dir() function behaves rather differently with different type of objects, as it aims to produce the most relevant one, rather than the complete information.
  - For Class Objects, it returns a list of names of all the valid attributes and base attributes as well.
  - For Modules/Library objects, it tries to return a list of names of all the attributes, contained in that module.
  - If no parameters are passed it returns a list of names in the current local scope.

- **Example:**

```
# let's import two modules
import random
import math
print(dir())
```

Output :

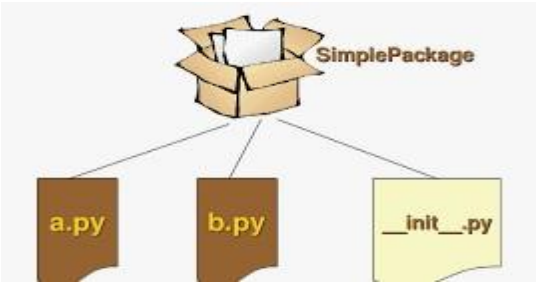
['__builtins__', '__cached__', '__doc__', '__file__','__loader__', '_name_', '_package_', '_spec_']

['__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__', 'math', 'random']

****

Packages:

- A package is a directory on the file system which contains a file named __init__.py.
- **__init__.py:**
  - It makes modules in the directory "importable".
  - __init__.py is empty. Or, just include a comment.
  - It is evaluated the first time that an application imports anything from that directory/package.
  - Any code that should be executed exactly once and during import.



- For example:
  - Perform initialization needed by the package.
  - Make variables, functions, classes, etc available. For example, when the package is imported rather than modules in the package.
- We can also expose objects defined in modules contained in the package.
  - Define a variable named __all__to specify the list of names that will be imported by

    ```
    from my_package import *
    ```
  - For example, if the following is present in my_package/__init__.py:
    ```
    __all__= ['func1', 'func2',]
    ```
  - Then, from my_package import * will import func1 and func2, but not other names defined in my_package. Note that__all__can be used at the module level, as well as at the package level.

:

Composition:

- In composition, we do not inherit from the base class but establish relationships between classes through the use of instance variables that are references to other objects. Talking in terms of pseudocode you may say that

```
class GenericClass:
      define some attributes and methods

class ASpecificClass:
      Instance_variable_of_generic_class = GenericClass

# use this instance somewhere in the class
      some_method(Instance_variable_of_generic_class)
```

- So you will instantiate the base class and then use the instance variable for any business logic.
- **How to achieve composition in Python:** To achieve composition you can instantiate other objects in the class and then use those instances. For example in the below example we instantiate the Rocket class using self.rocket and then using self.rocket in the method get_maker.

```
class MarsRoverComp():
    def___init__(self, name, distance, maker):
        self.rocket = Rocket(name, distance) # instantiating the base
        self.maker = maker
    def get_maker(self):
        return "%s Launched by %s" % (self.rocket.name, self.maker)

if___name___== "__main__":
    z = MarsRover("mars_rover2", "till Mars", "ISRO")
    print(z.launch())
    print(z.get_maker())
```

- The output of the total code which has both inheritance and composition is shown below:

➜ Documents python rockets.py

```
simple rocket has reached till stratosphere
mars_rover has reached till Mars
mars_rover Launched by ISRO
mars_rover2 has reached till Mars
mars_rover2 Launched by ISRO
```

****

Standard Library:

- Python's standard library is very extensive, offering a wide range of facilities as indicated by the long table of contents listed below.
- The library contains built-in modules (written in C) that provide access to system functionality such as file I/O that would otherwise be inaccessible to Python programmers, as well as modules written in Python that provide standardized solutions for many problems that occur in everyday programming.
- Some of these modules are explicitly designed to encourage and enhance the portability of Python programs by abstracting away platform-specifics into platform-neutral APIs.
- The Python Standard Library is a collection of script modules accessible to a Python program to simplify the programming process and removing the need to rewrite commonly used commands. They can be used by 'calling/importing' them at the beginning of a script.
- A list of the Standard Library modules can be found at http://www.python.org/doc/.
- The following are among the most important:
    - time, sys, os, math, random, pickle, urllib, re, cgi, socket
    - numpy,matplotlib, scipy
- Python standard library provides
    - Built-in Functions, Built-in Constants, Built-in Types, Built-in Exceptions
    - Text Processing Services, Binary Data Services, Python Language Services, Unix Specific Services, Cryptographic Services, Generic Operating System Services, Multimedia Services, Python Runtime Services, Miscellaneous Services
    - File and Directory Access, File Formats, Data Persistence, Data Compression and Archiving
    - Importing Modules, Numeric and Mathematical Modules, Functional Programming Modules
    - Concurrent Execution, Inter process Communication and Networking, Internet Data Handling
    - Structured Markup Processing Tools, Internet Protocols and Support
    - Graphical User Interfaces with Tk.
    - Program Frameworks, Development Tools, Debugging and Profiling, Custom Python Interpreters

****